VENCIMIENTO DE DESARROLLAR SOFTWARE A PARTIR DE LA INTEGRACIÓN DE SUS MÓDULOS COMPONENTES. GA8-220501096-AA1-EV01.

Integrantes:

Juan Jose Benavides Ruiz

Escarleth Yisel Rios

Carolina Monroy Giraldo

Análisis y Desarrollo de Software

No. Ficha: 3336133

Instructor:

Denis Alfredo Colunge Belalcazar

Centro de Formación:

Centro de la construcción

07/11/2026

INTRODUCCIÓN

El presente documento describe el desarrollo e integración de los módulos principales del sistema web Moto Repuestos Express JYC. La aplicación permite consultar productos, filtrarlos por categoría, registrar usuarios, iniciar sesión, administrar un carrito de compras y registrar pedidos.

La versión actual utiliza un frontend desarrollado con React y Vite, un backend construido con Spring Boot y Maven, y una base de datos MySQL. Los módulos fueron integrados mediante una API REST y se verificaron mediante pruebas funcionales, pruebas de API en Postman y pruebas unitarias automatizadas.

En este contexto, la integración no consiste únicamente en conectar las pantallas con el backend. También implica comprobar que los componentes intercambien datos con el formato esperado, respeten los contratos de las rutas, manejen los errores y mantengan la información consistente en la base de datos. Por esta razón, se verificó el recorrido de la información desde React hacia la API REST, las capas de Spring Boot y finalmente MySQL[1].

El documento también presenta la arquitectura implementada, la organización del código fuente, las configuraciones de los ambientes, los mecanismos de seguridad, los archivos compilados y los resultados obtenidos durante las pruebas.

OBJETIVO GENERAL.

Desarrollar, integrar y validar los módulos funcionales de la aplicación web Moto Repuestos Express JYC, utilizando React, Spring Boot, Maven y MySQL, de acuerdo con los requerimientos, artefactos de diseño y buenas prácticas de desarrollo de software.

OBJETIVOS ESPECÍFICOS

- ✓ Integrar el frontend React con la API REST desarrollada en Spring Boot.
- ✓ Implementar los módulos de productos, usuarios, carrito y pedidos.
- ✓ Conectar el backend con la base de datos MySQL.
- ✓ Aplicar una arquitectura organizada por capas y componentes reutilizables.
- ✓ Validar los datos recibidos por los servicios de la aplicación.
- ✓ Proteger las contraseñas mediante cifrado BCrypt.
- ✓ Ejecutar pruebas unitarias, pruebas de API y pruebas de integración.
- ✓ Generar los archivos compilados del frontend y del backend.
- ✓ Documentar el ambiente de desarrollo, las configuraciones y los resultados.

ALCANCE

La versión implementada permite consultar y filtrar productos, registrar usuarios, iniciar sesión mediante correo electrónico, administrar el carrito de compras, crear pedidos y gestionar productos mediante operaciones de consulta, creación, actualización y eliminación.

La aplicación implementa autenticación básica mediante correo y contraseña cifrada con BCrypt. La autorización avanzada por roles y la autenticación mediante tokens JWT se consideran mejoras futuras y no se presentan como funcionalidades implementadas en esta versión.

ANTECEDENTES

Los modelos conceptual y lógico elaborados en las evidencias anteriores constituyeron la base inicial para identificar las entidades y relaciones del sistema.

Durante las etapas posteriores, el proyecto evolucionó desde el diseño de datos hacia una aplicación web integrada. Por esta razón, los diagramas actuales fueron ajustados para representar la arquitectura implementada con React, Spring Boot, Maven y MySQL.

Los diagramas anteriores se conservan como antecedentes del análisis, mientras que los incluidos en esta evidencia representan la versión actual y funcional del sistema.

Los artefactos iniciales contemplaban un alcance más amplio, con entidades como roles, clientes, administradores, categorías, carrito, pedidos, detalles de pedido y pagos. En la implementación actual se priorizaron las entidades producto, usuario, pedido y detalle_pedido, necesarias para validar el flujo principal de la aplicación.

La versión actual no incluye recuperación automática de contraseña ni envío de correos. Esta delimitación permitió entregar primero un producto funcional y verificable; las funcionalidades y entidades restantes podrán incorporarse posteriormente sin modificar la arquitectura general del sistema.

REQUERIMIENTOS FUNCIONALES IMPLEMENTADOS

Los requerimientos funcionales se organizaron por módulos para facilitar la trazabilidad entre las necesidades del sistema, la codificación realizada y las pruebas ejecutadas.

Código	Módulo	Requerimiento funcional	Evidencia de validación	Estado
RF-01	Productos	El sistema debe permitir consultar y filtrar los productos disponibles por categoría.	Consulta en React y solicitud GET /api/productos.	Implementado
RF-02	Productos	El sistema debe permitir crear, actualizar y eliminar productos.	Pruebas POST, PUT y DELETE en Postman.	Implementado

RF-03	Usuarios	El sistema debe permitir registrar usuarios con datos obligatorios, correo válido y contraseña segura.	Registro desde React, Postman y validación 400 Bad Request.	Implementado
RF-04	Usuarios	El sistema debe permitir iniciar sesión mediante correo y contraseña.	Prueba de login exitoso y rechazo de credenciales incorrectas.	Implementado
RF-05	Carrito	El sistema debe permitir agregar y eliminar productos y calcular el total de compra.	Prueba funcional del carrito en React.	Implementado
RF-06	Pedidos	El sistema debe permitir crear pedidos con los productos seleccionados.	Confirmación en React y respuesta 201 Created.	Implementado
RF-07	Inventario	El sistema debe descontar del stock la cantidad incluida en un pedido.	Verificación del pedido y del inventario en MySQL.	Implementado
RF-08	Administración	El sistema debe permitir gestionar los productos mediante una interfaz administrativa.	Página de administración y pruebas CRUD de la API.	Implementado

METODOLOGÍA DE DESARROLLO

Para el desarrollo del proyecto se aplicó un enfoque incremental e iterativo. Primero se revisaron los requerimientos y los modelos elaborados en las evidencias anteriores. Luego se organizaron los módulos, se implementó cada funcionalidad, se integraron frontend, backend y base de datos, y finalmente se ejecutaron pruebas para validar el comportamiento del sistema.

El trabajo se dividió en las siguientes etapas:

1. Revisión de requerimientos y artefactos anteriores.
2. Definición de la arquitectura y las tecnologías.
3. Implementación de los módulos del backend.
4. Implementación de las páginas y componentes del frontend.
5. Integración con MySQL mediante la API REST.
6. Pruebas unitarias, pruebas de API y pruebas funcionales.
7. Compilación, documentación y control de versiones.

HERRAMIENTAS Y AMBIENTES DE DESARROLLO

Herramienta	Uso en el proyecto
Visual Studio Code	Edición y organización del código fuente
MySQL Workbench	Creación, consulta y verificación de la base de datos
Postman	Pruebas de los endpoints de la API
PowerShell	Ejecución del backend, pruebas y compilación
Maven Wrapper	Gestión y ejecución del proyecto Spring Boot
Git y GitHub	Control de versiones y almacenamiento del código
Navegador web	Validación funcional del frontend

ARQUITECTURA DEL SISTEMA

La solución utiliza una arquitectura por capas. El frontend contiene las páginas, componentes reutilizables, servicios HTTP y el estado global del carrito. El backend expone controladores REST, aplica la lógica de negocio en los servicios y utiliza repositorios JPA y JDBC para acceder a MySQL.

Los diagramas asociados se encuentran en la carpeta Diagramas:

1. Arquitectura general.
2. Diagrama de componentes.
3. Diagrama de paquetes.
4. Diagrama de clases.
5. Diagrama de casos de uso.

6. Mapa de navegación.

TECNOLOGÍAS, LIBRERÍAS Y FRAMEWORKS.

Tecnología	Uso
Java 21	Lenguaje del backend
Spring Boot	Framework del backend y API REST
Maven	Gestión de dependencias y compilación
Spring Web MVC	Controladores y servicios HTTP
Spring Data JPA	Persistencia de entidades
Spring JDBC	Consultas SQL del módulo de productos
Spring Validation	Validación de datos de entrada
Spring Security Crypto	Cifrado BCrypt de contraseñas
MySQL Connector	Conexión con MySQL
React y Vite	Desarrollo del frontend
React Router	Navegación de la aplicación
Git y GitHub	Control de versiones
Postman	Pruebas de la API REST

ESTRUCTURA DE LAS CARPETAS

MotoRepuestosExpress/

├backend-spring/

│ └─src/main/java/com/motorepuestos/backend_spring/

│ ├ controller/

│ ├ model/

│ ├ repository/

│ └ service/

| |__ ProyectoReact/

│ └── src/

│ |__components/

│ |__ context/

│ |__ pages/

│ |__ services/

├IMG/

├ Diagramas/

└- Evidencias/

SEGURIDAD Y VALIDACIONES

La seguridad y las validaciones permiten proteger la información de los usuarios, evitar datos incorrectos y controlar la comunicación entre el frontend y el backend.

Cuando un usuario se registra, React envía los datos mediante la solicitud:

POST /api/usuarios/registro

El UsuarioController recibe la solicitud y aplica la validación mediante @Valid. Spring verifica que:

- El nombre no esté vacío.
- El apellido y el documento sean obligatorios.
- El correo tenga un formato válido.
- La contraseña tenga mínimo ocho caracteres.

Después, UsuarioService verifica que el correo y el nombre de usuario no estén registrados anteriormente. Si los datos son válidos, BCrypt cifra la contraseña y el repositorio guarda el usuario en MySQL.

La contraseña no se almacena con su valor real. Por ejemplo, el sistema no guarda directamente:

Moto12345

En su lugar, guarda una representación cifrada. BCrypt no permite recuperar la contraseña original. Cuando el usuario inicia sesión, React envía el correo y la contraseña al backend. Spring busca el usuario mediante el correo y BCrypt compara la contraseña ingresada con el valor cifrado almacenado.

Si las credenciales coinciden, el sistema devuelve un inicio de sesión exitoso. Si no coinciden, responde:

401 Unauthorized

También se realizaron pruebas con datos incorrectos para comprobar que las validaciones funcionaran. Por ejemplo, se envió un registro sin nombre, con un correo inválido y una contraseña demasiado corta. La API rechazó la solicitud con:

400 Bad Request

En el módulo de productos se validan las siguientes condiciones:

- El nombre es obligatorio.
- La categoría es obligatoria.
- El precio debe ser mayor que cero.
- El stock debe ser igual o mayor que cero.

Finalmente, CORS controla la comunicación entre el frontend y el backend. Actualmente permite solicitudes desde:

http://localhost:5173

Esto permite que React consuma la API de Spring Boot, evitando aceptar conexiones abiertas desde cualquier origen. Además, la contraseña de MySQL se obtiene mediante la variable de entorno DB_PASSWORD y no se publica en el repositorio.

CONFIGURACIÓN DE AMBIENTES

Base de datos

Motor: MySQL

Base de datos: motorepuestos

Puerto: 3306

Tablas principales: producto, usuario, pedido, detalle_pedido

Backend

Framework: Spring Boot

Puerto: 8080

URL: http://localhost:8080

Frontend

Framework: React/Vite

Puerto: 5173

URL: http://localhost:5173

INTEGRACIÓN DE MODULOS

El frontend React consulta la API REST mediante solicitudes HTTP en formato JSON. Spring Boot procesa las solicitudes, aplica las reglas de negocio y consulta o actualiza los registros de MySQL.

El flujo principal de integración es:

Usuario → React → API REST Spring Boot → Servicios → Repositorios → MySQL

Este flujo fue comprobado mediante la consulta de productos, el registro e inicio de sesión, la creación del carrito y el registro de pedidos.

PRUEBAS REALIZADAS

Prueba	Resultado esperado	Resultado obtenido
Consulta de productos	Lista de productos	200 OK
Registro de usuario	Usuario creado	201 Created
Inicio de sesión correcto	Acceso permitido	200 OK
Credenciales incorrectas	Acceso rechazado	401 Unauthorized
Registro con datos inválidos	Solicitud rechazada	400 Bad Request
Crear producto	Producto almacenado	Respuesta exitosa
Consultar producto	Producto encontrado	Respuesta exitosa
Actualizar producto	Datos modificados	Respuesta exitosa
Eliminar producto de prueba	Registro eliminado	Respuesta exitosa
Crear pedido	Pedido almacenado	201 Created
Pruebas automatizadas	Sin fallos	12 pruebas exitosas

EVIDENCIAS DE EJECUCIÓN

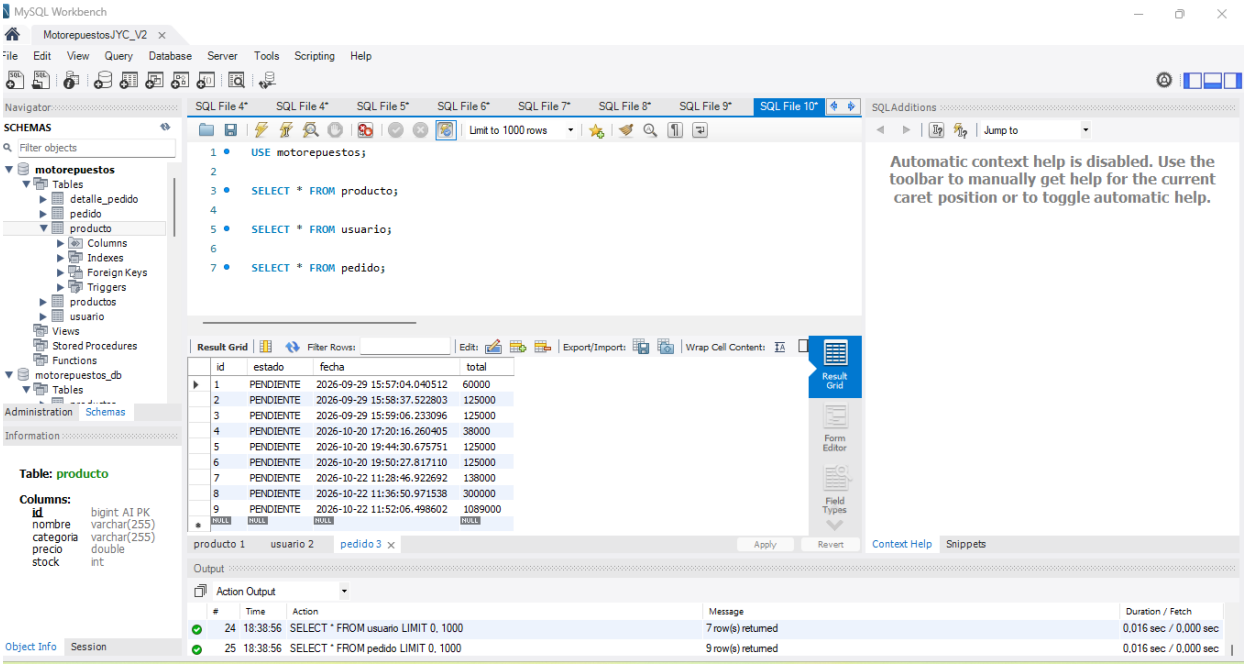


Figura 1. Conexión y tablas de MySQL



Figura 2. API Spring consultando productos

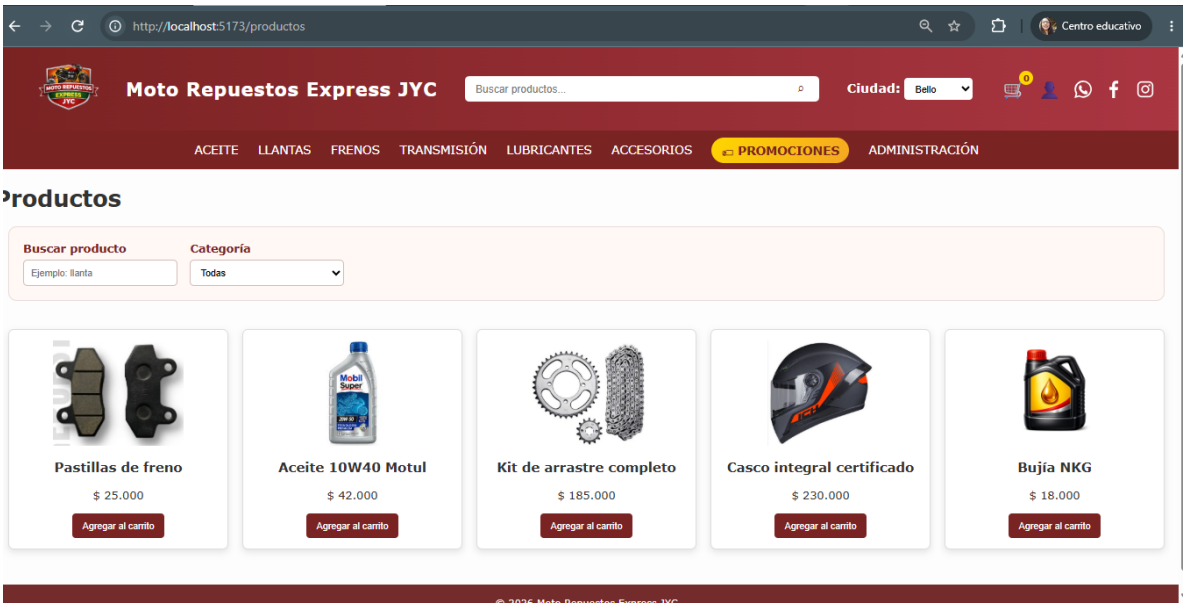


Figura 3. Productos mostrados en React



Figura 4. Producto agregado al carrito

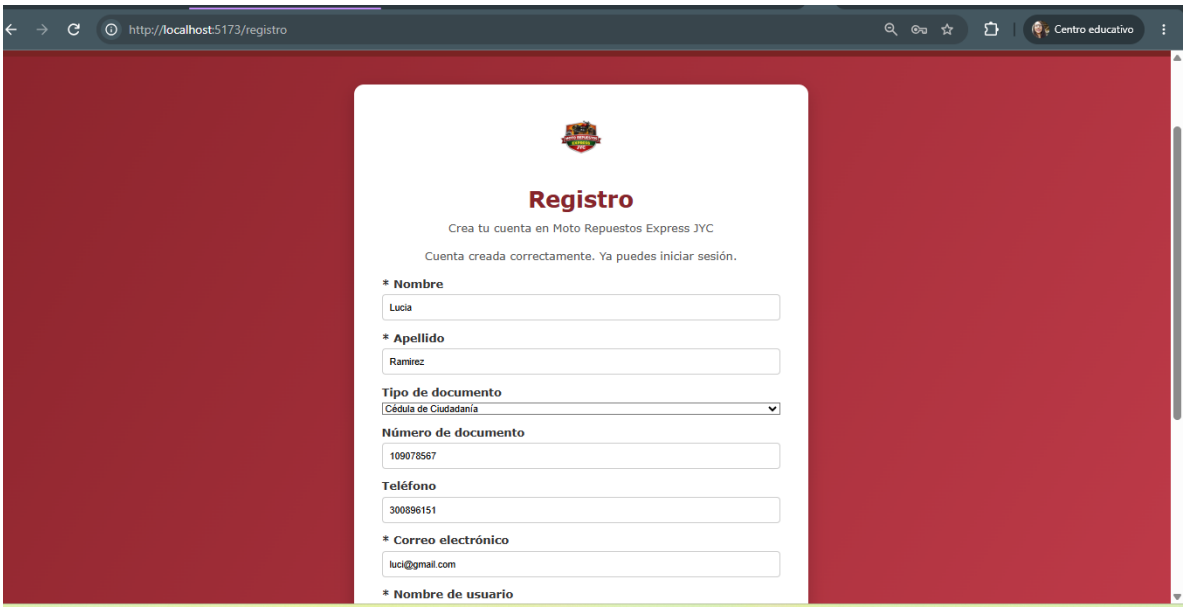


Figura 5. Registro usuario exitoso

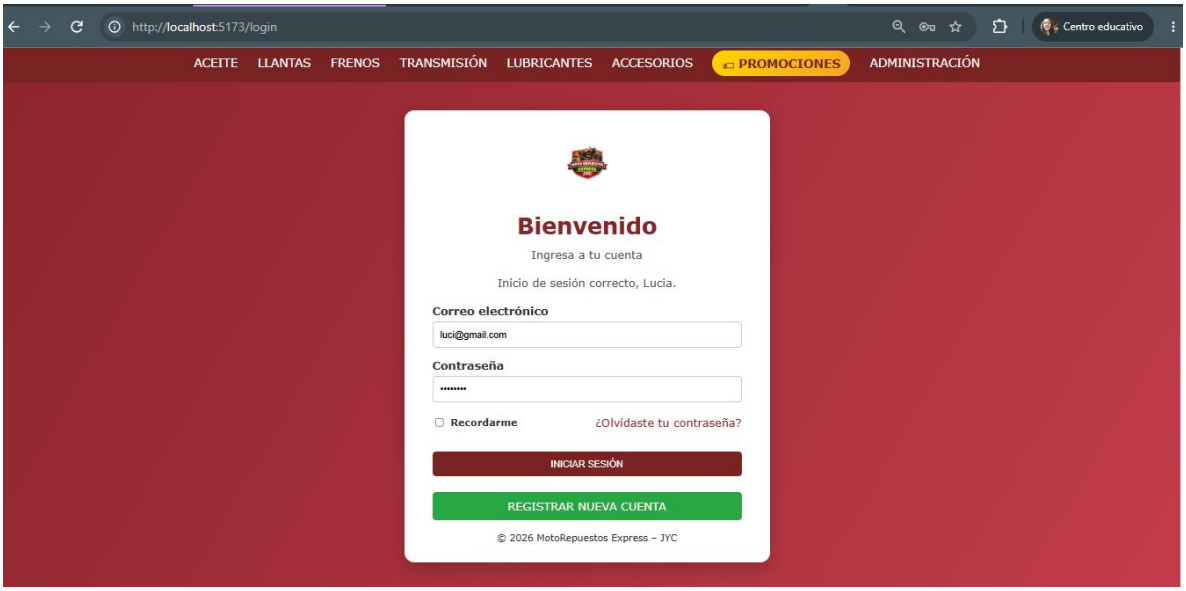


Figura 6. Inicio de sesión exitoso

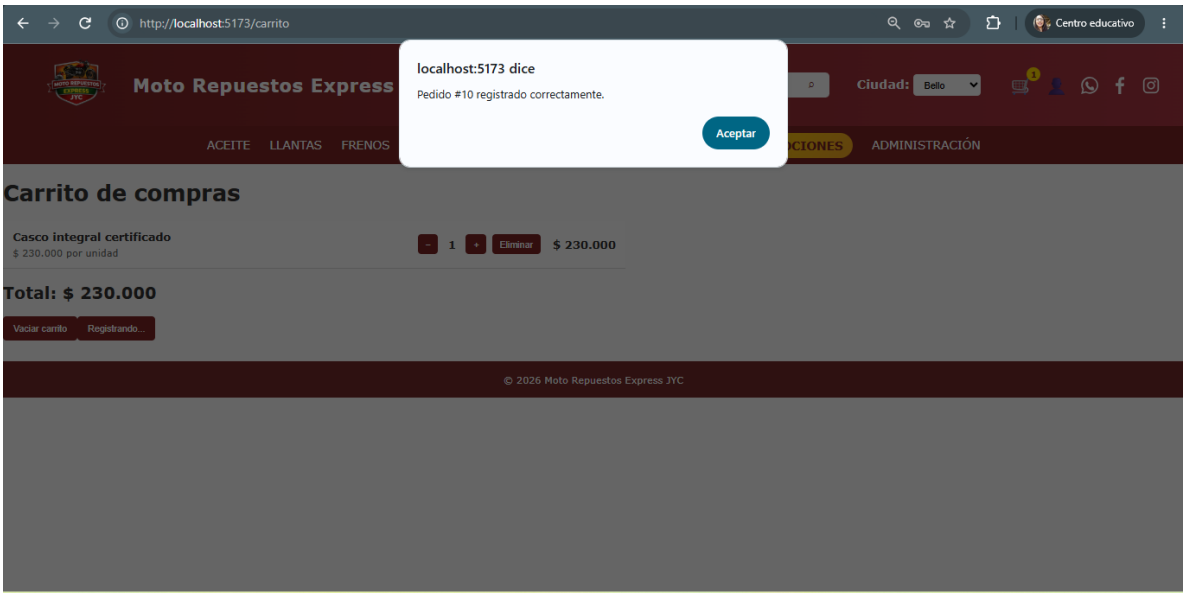


Figura 7. Confirmación de pedido.

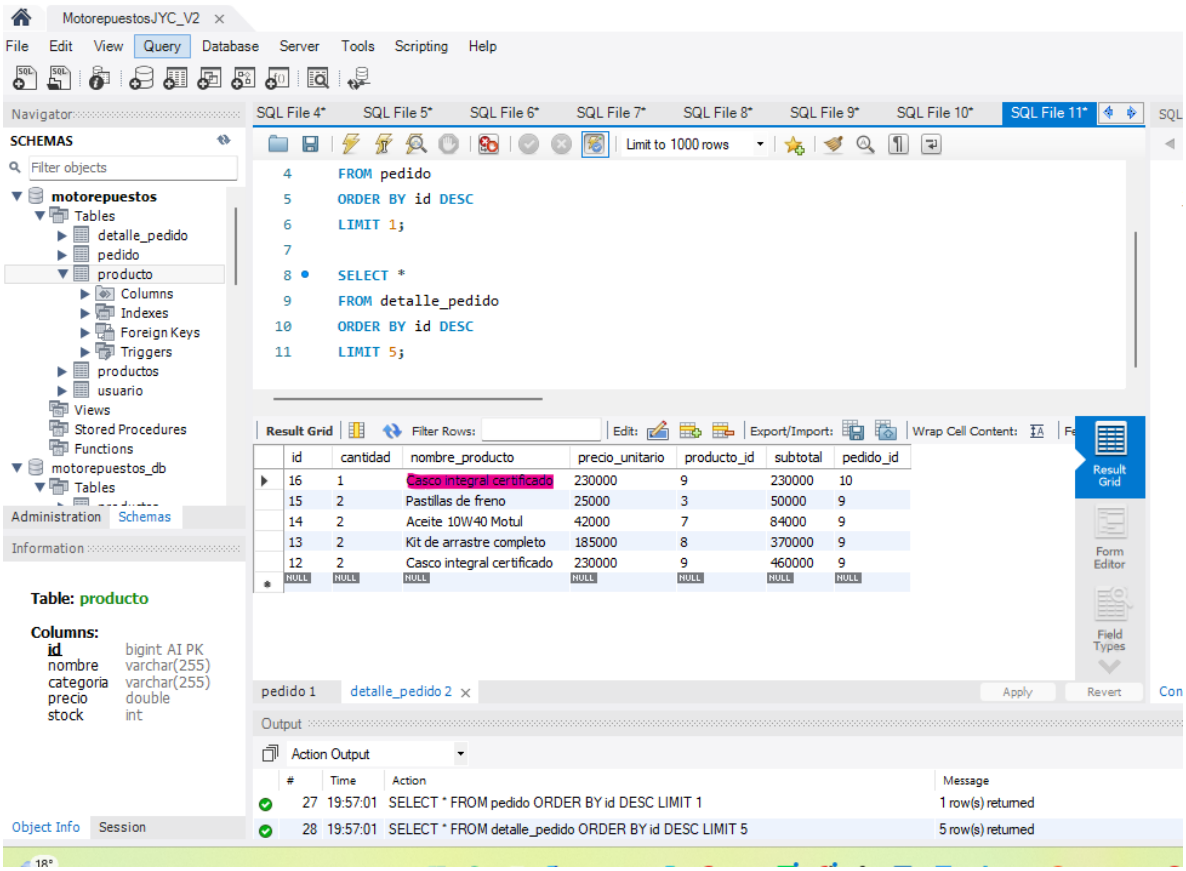


Figura 8. Pedido almacenado en MySQL

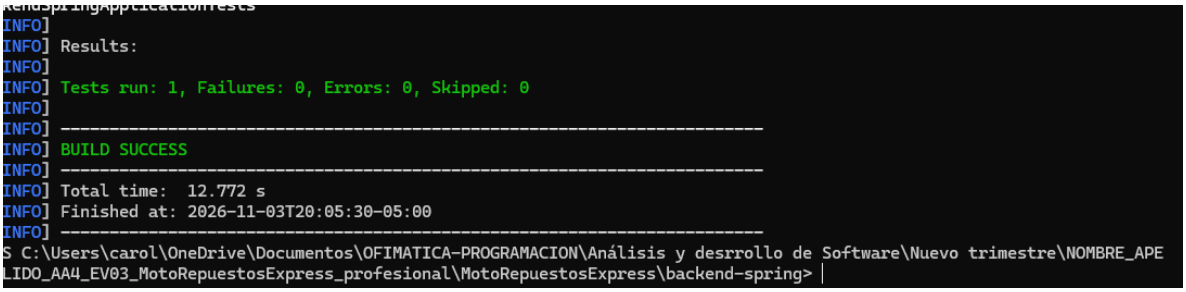


Figura 9. Prueba automatizada inicial

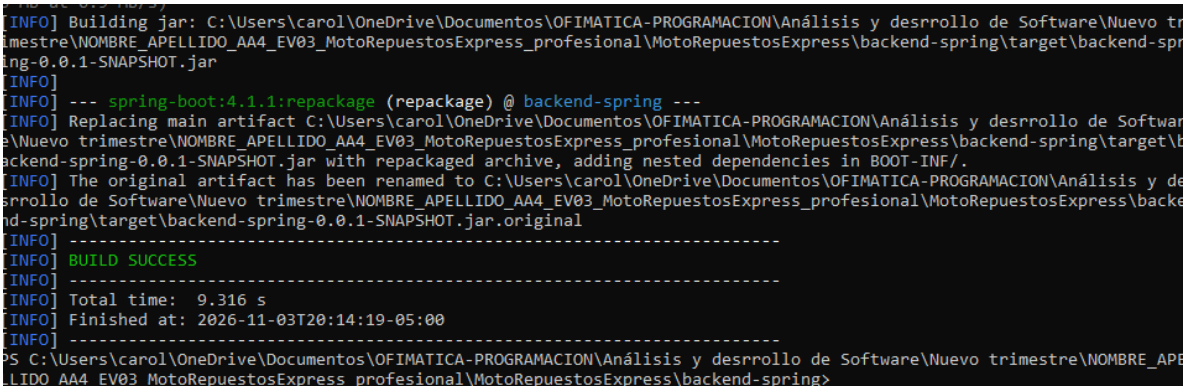


Figura 10. Generación del archivo JAR

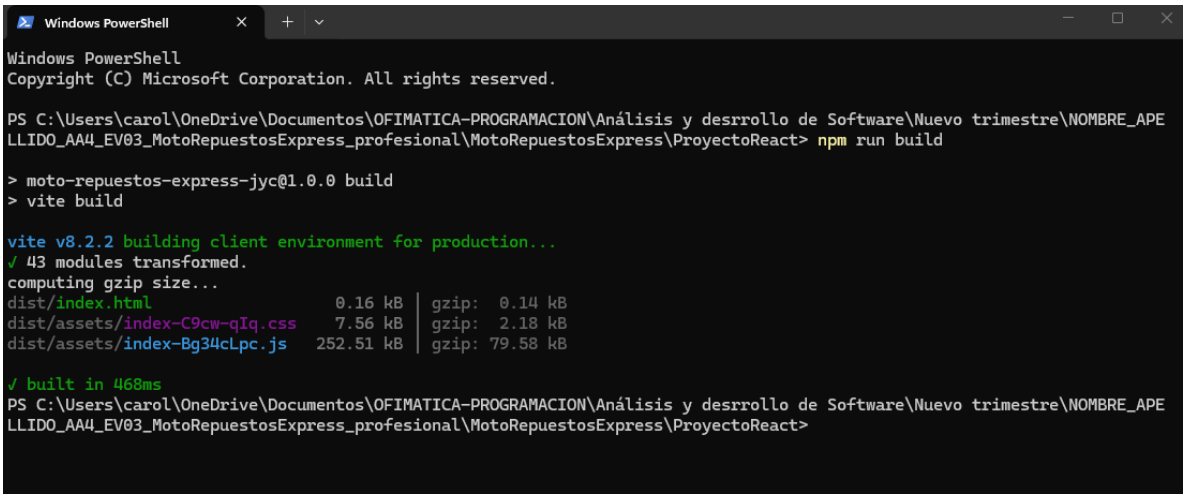


Figura 11. Compilación de React

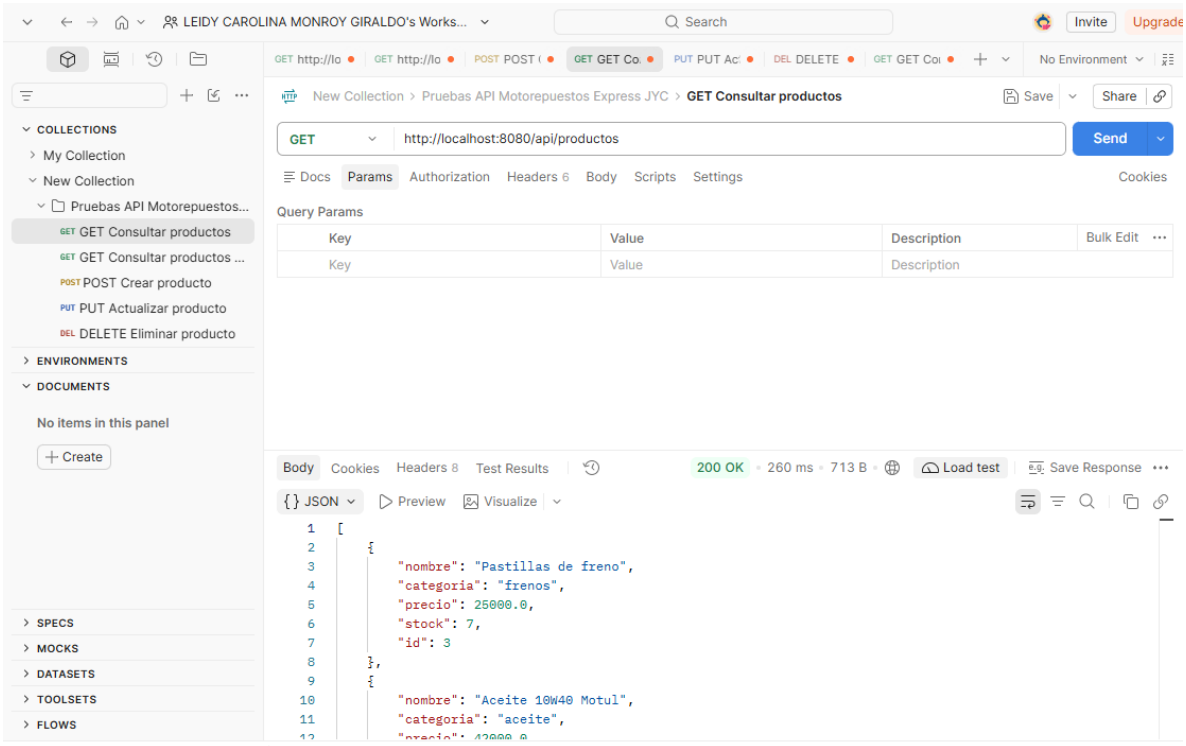


Figura 12. Postman GET consultar productos

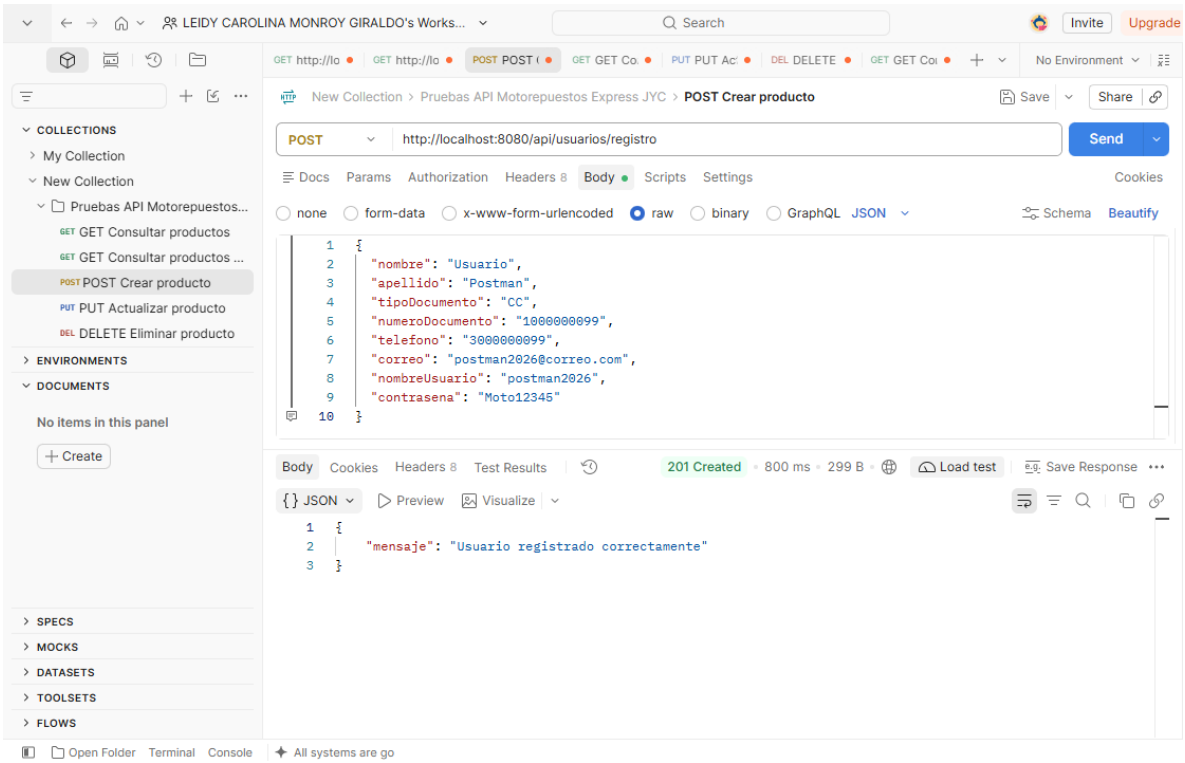


Figura 13. Postman POST crear usuario

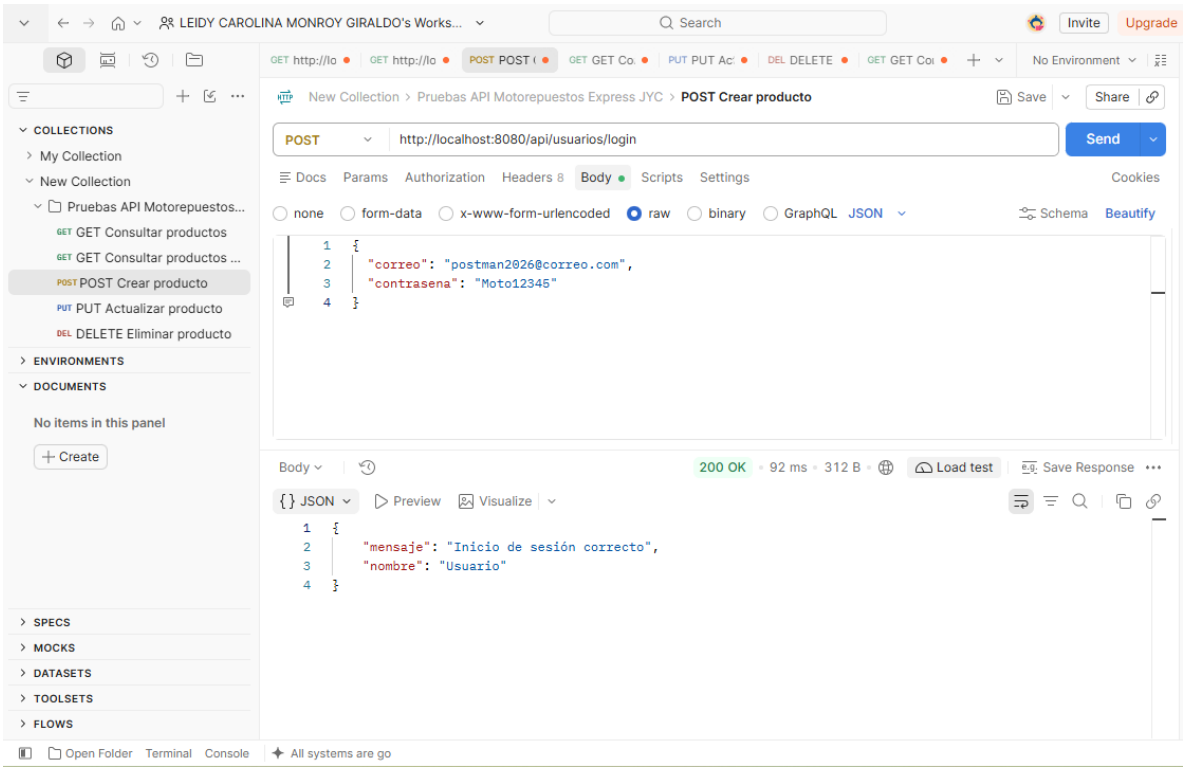


Figura 14. Postman POST login exitoso

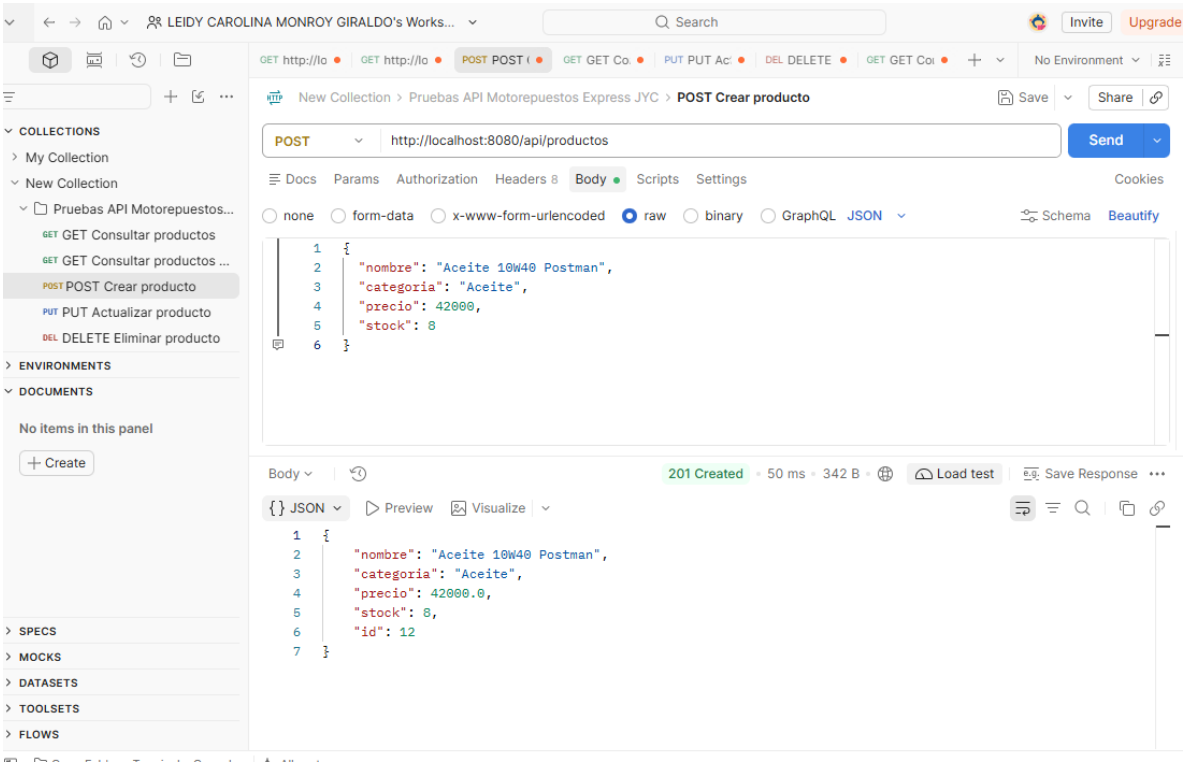


Figura 15. Postman POST crear producto

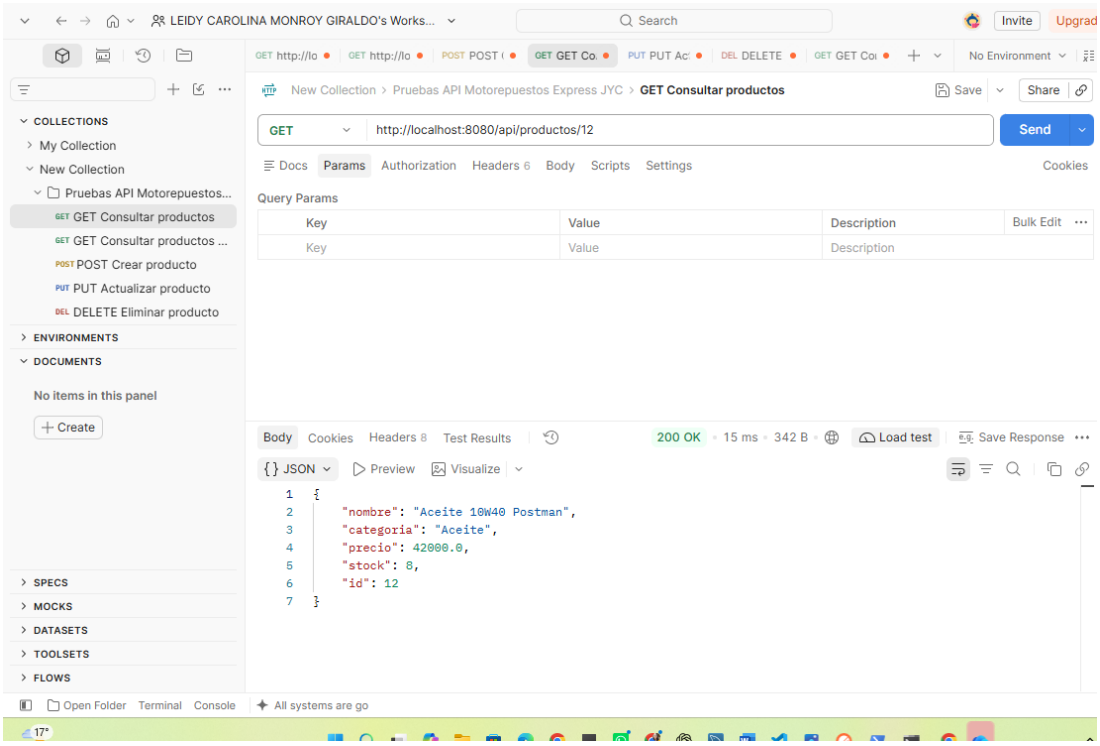


Figura 16. Postman GET consultar producto

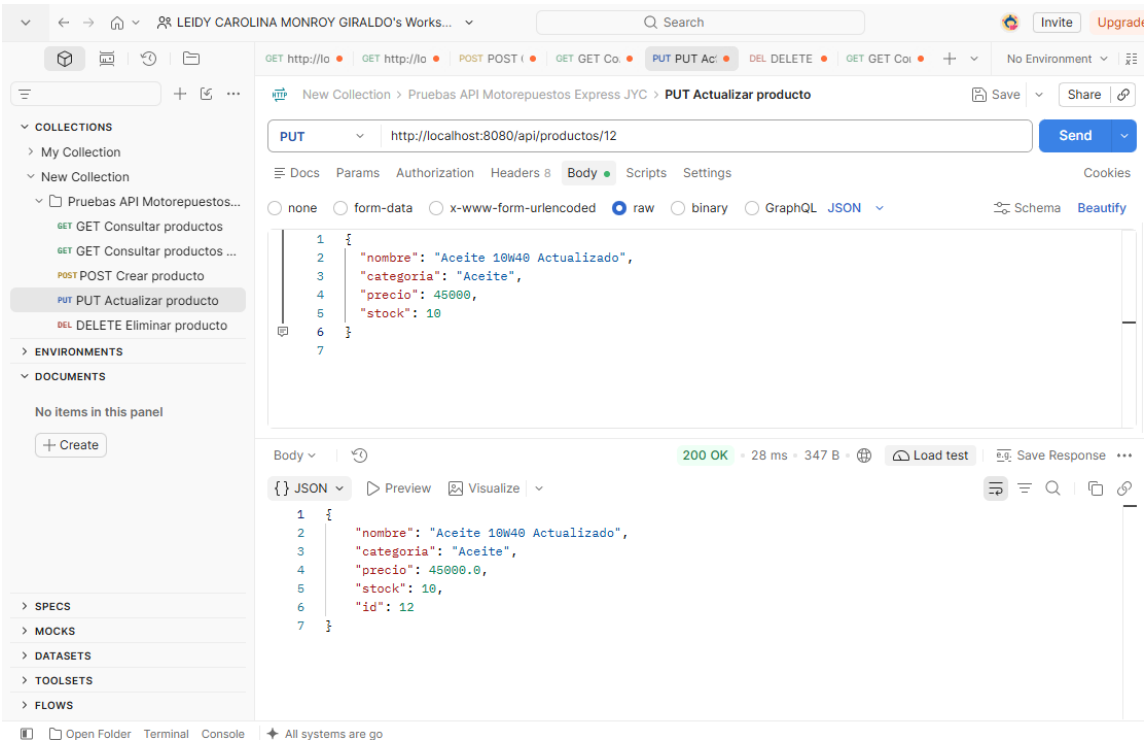


Figura 17. Postman PUT actualizar producto

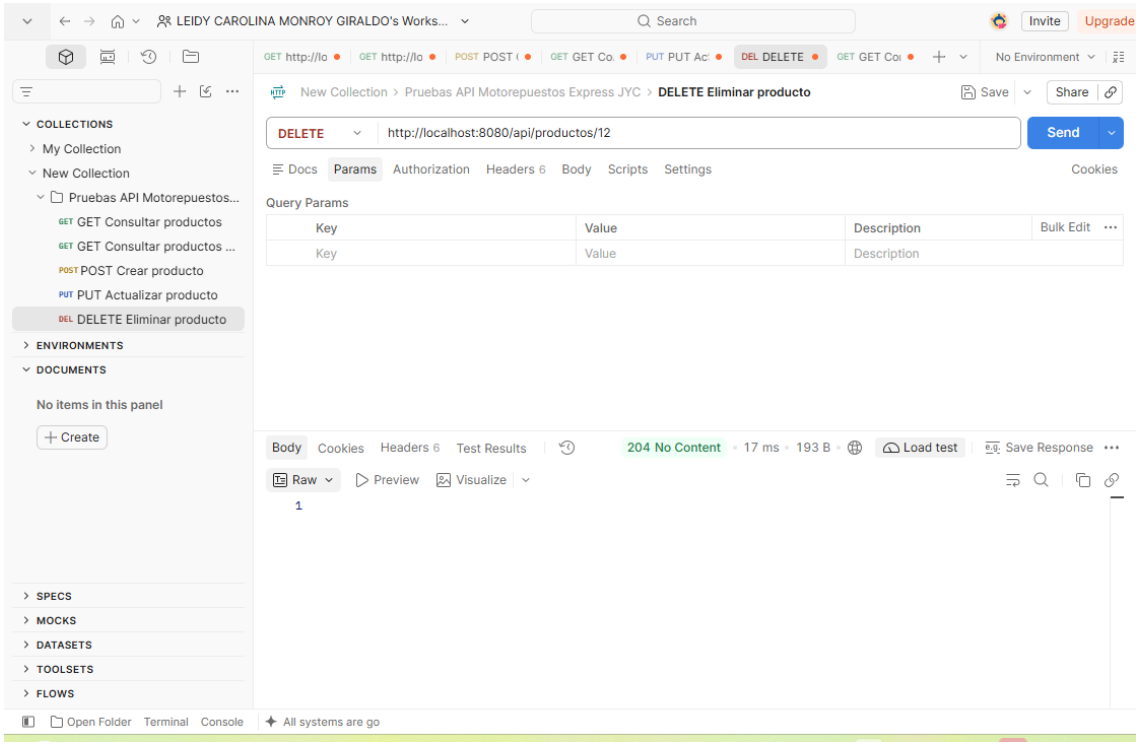


Figura 18. Postman DELETE eliminar producto

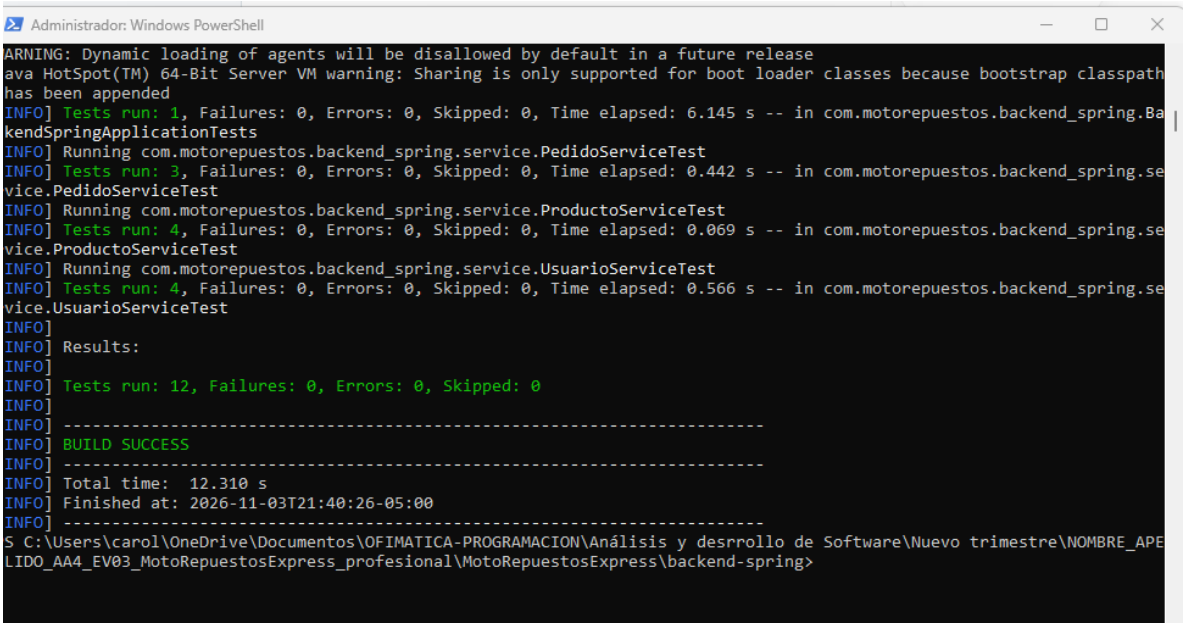


Figura 19. Doce pruebas automatizadas servicio Spring exitosas.

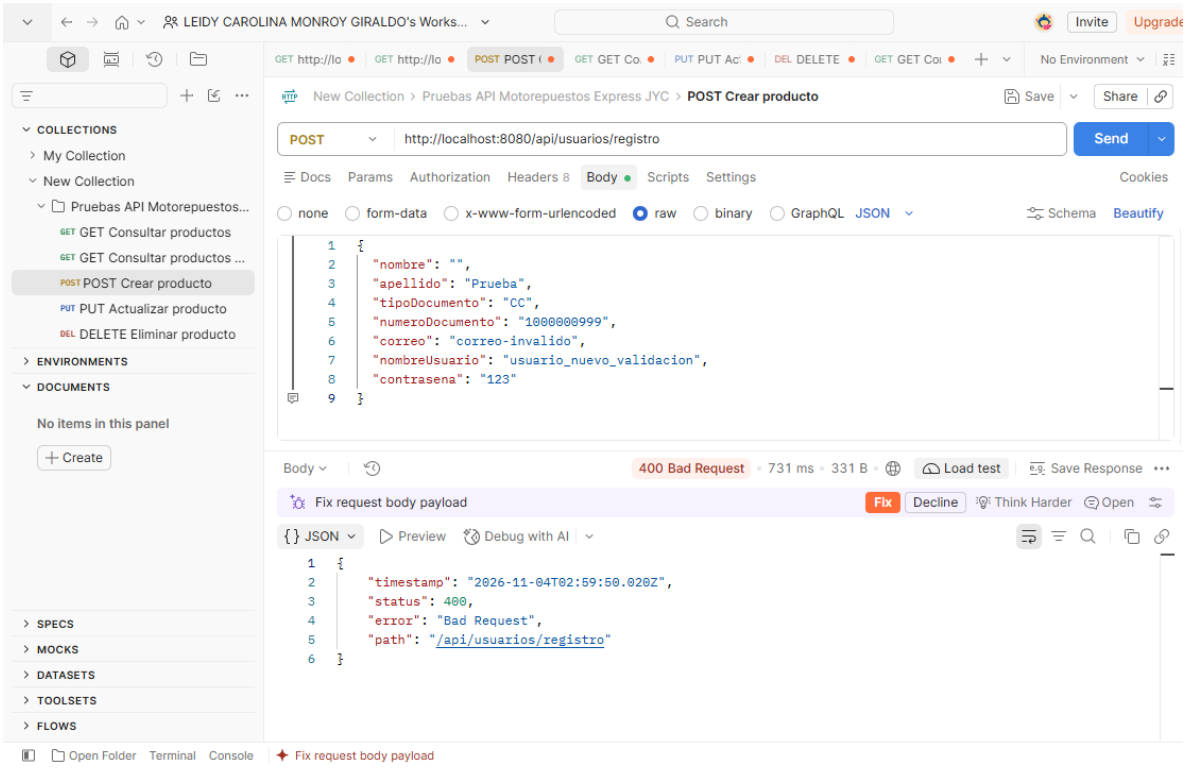


Figura 20. Postman POST Rechazo de datos inválidos.

ARCHIVOS COMPILADOS

El backend fue empaquetado correctamente mediante Maven y generó:

backend-spring/target/backend-spring-0.0.1-SNAPSHOT.jar

El frontend fue compilado mediante Vite y generó la carpeta:

ProyectoReact/dist/

CONTROL DE VERSIONES

Repositorio del proyecto:

https://github.com/lcarolinamonroy-create/MotoRepuestosExpressJYC

Ambiente local del frontend:

http://localhost:5173

API REST local:

http://localhost:8080/api/productos

La URL publicada del prototipo HTML anterior se conserva únicamente como antecedente y no representa la versión final integrada.

CONCLUSIONES

Se logró desarrollar e integrar los módulos principales de Moto Repuestos Express JYC mediante una arquitectura web basada en React, Spring Boot, Maven y MySQL.

Las pruebas realizadas permitieron comprobar el funcionamiento de la consulta y administración de productos, el registro y autenticación de usuarios, el carrito de compras y la creación de pedidos.

Además, se verificó la calidad básica del código mediante validaciones, pruebas unitarias, pruebas de API y generación de archivos compilados.

REFERENCIAS

1. Tutoriales Programación Ya. Pruebas de integración: qué significa integrar componentes de software. https://tutorialesprogramacionya.com/testing/pruebasdeintegracion/tema4.html

ANEXOS

Anexo A. Diagramas del sistema

Arquitectura del sistema

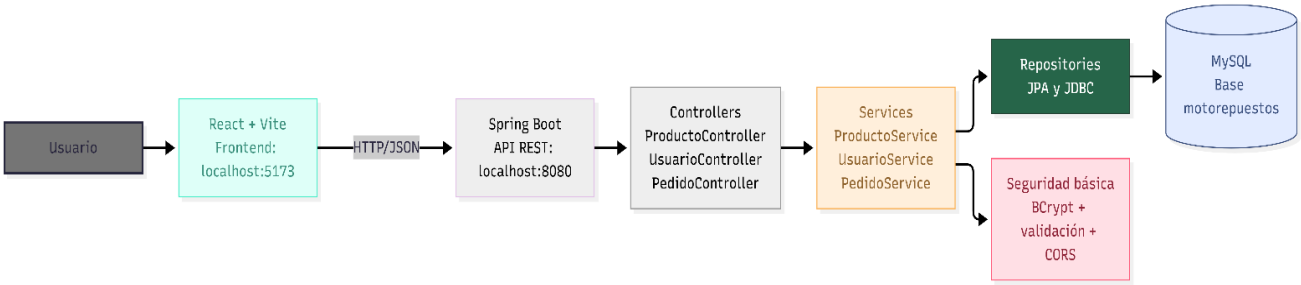


Diagrama de componentes

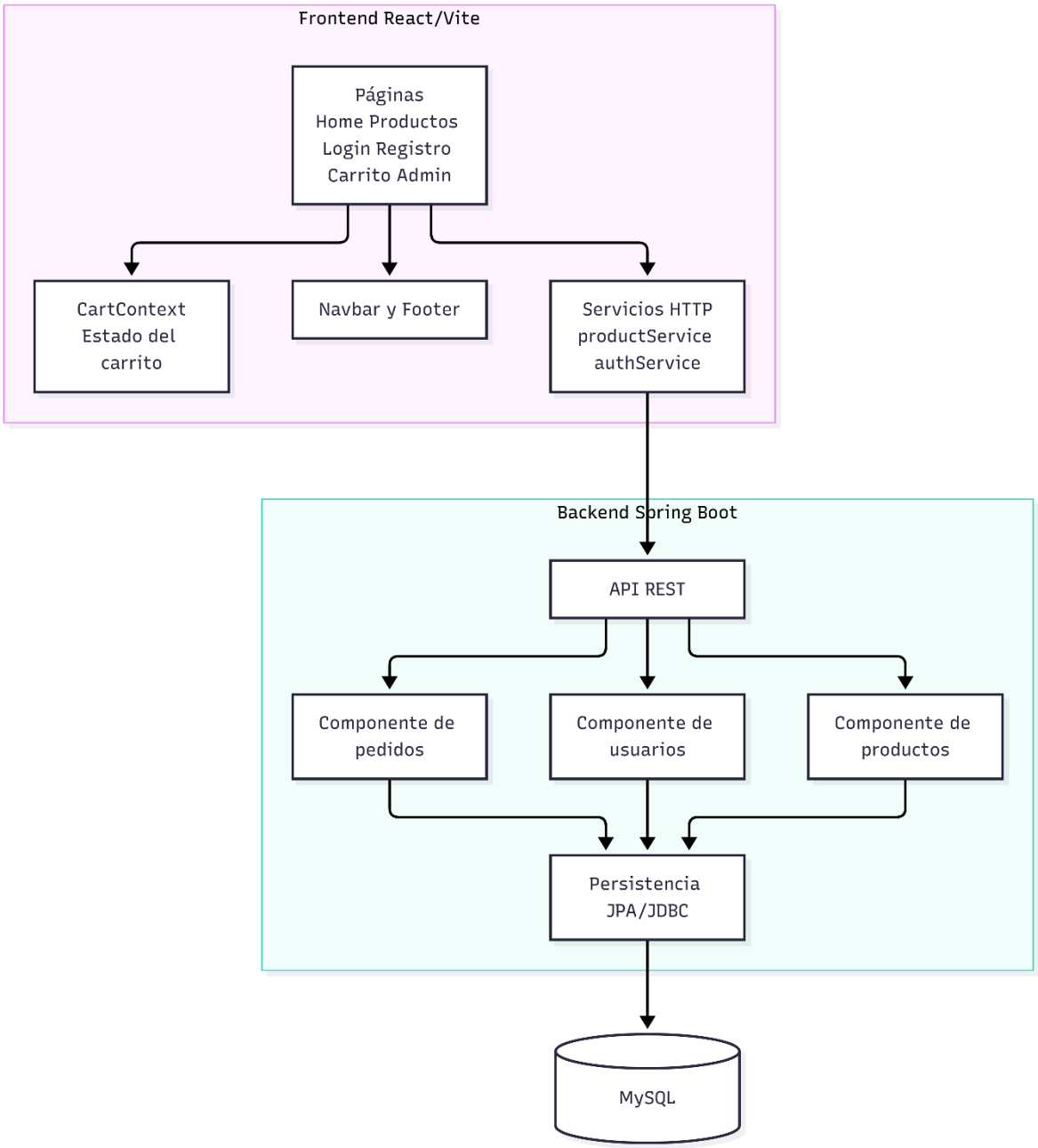


Diagrama de paquetes

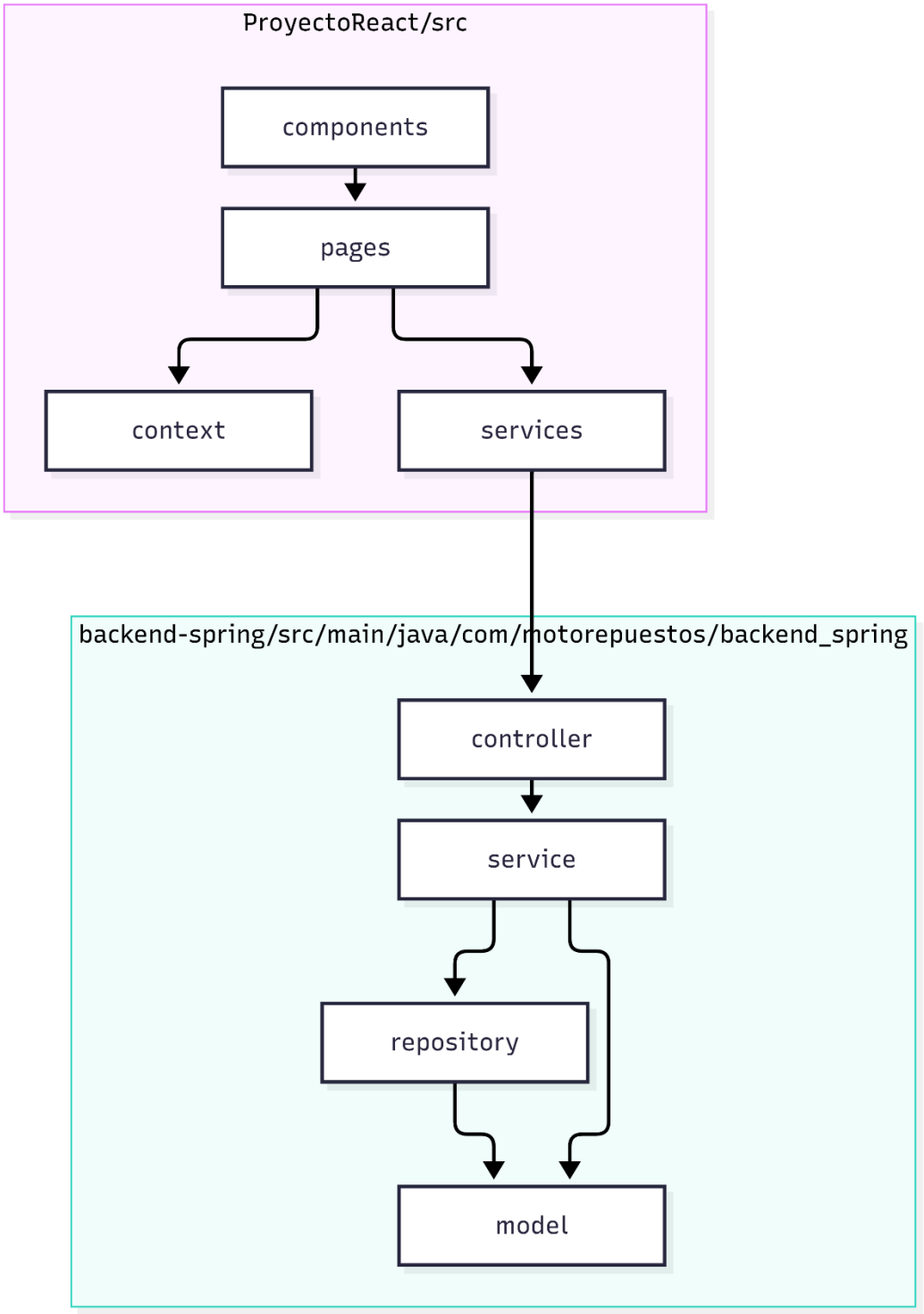
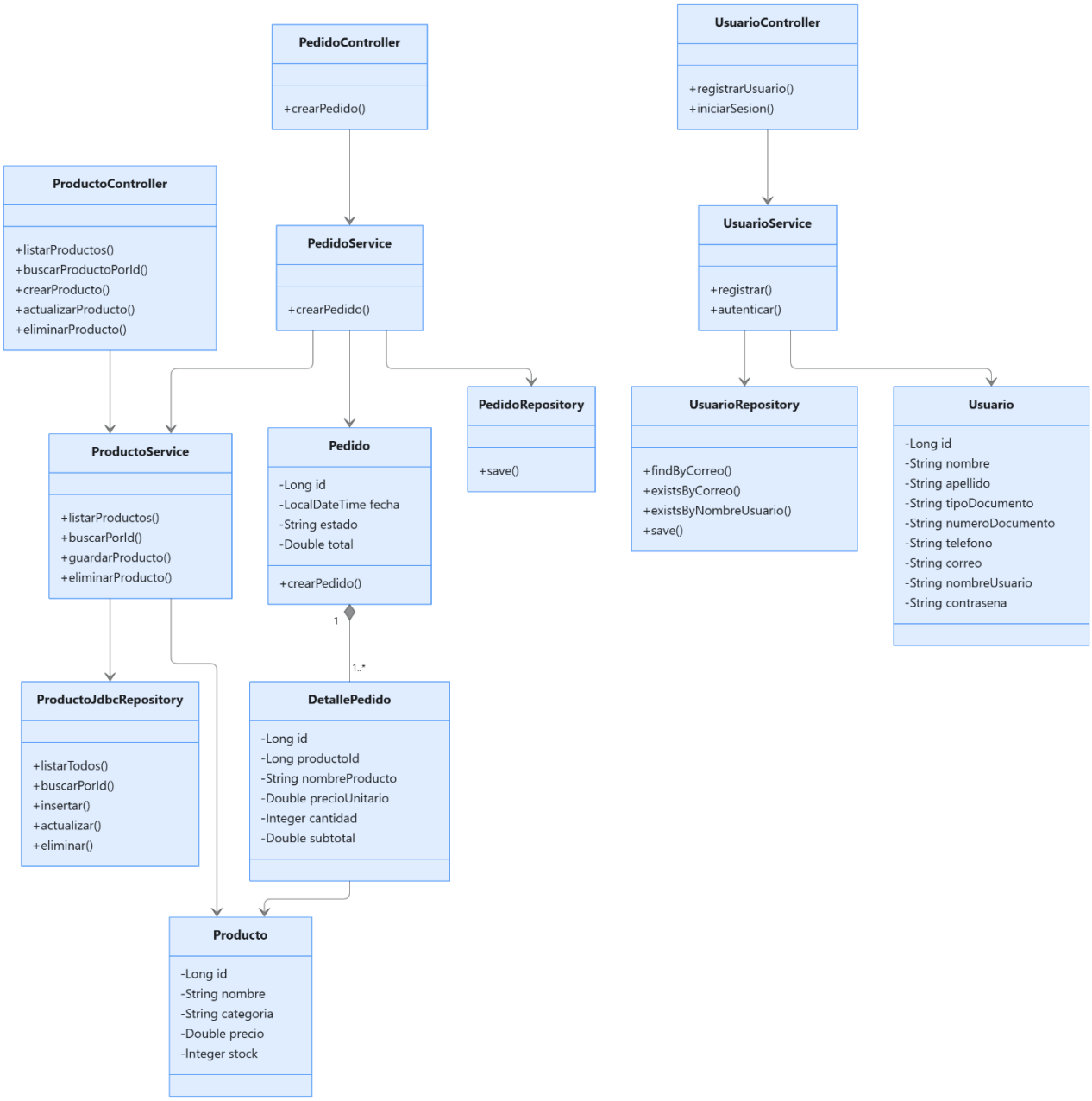
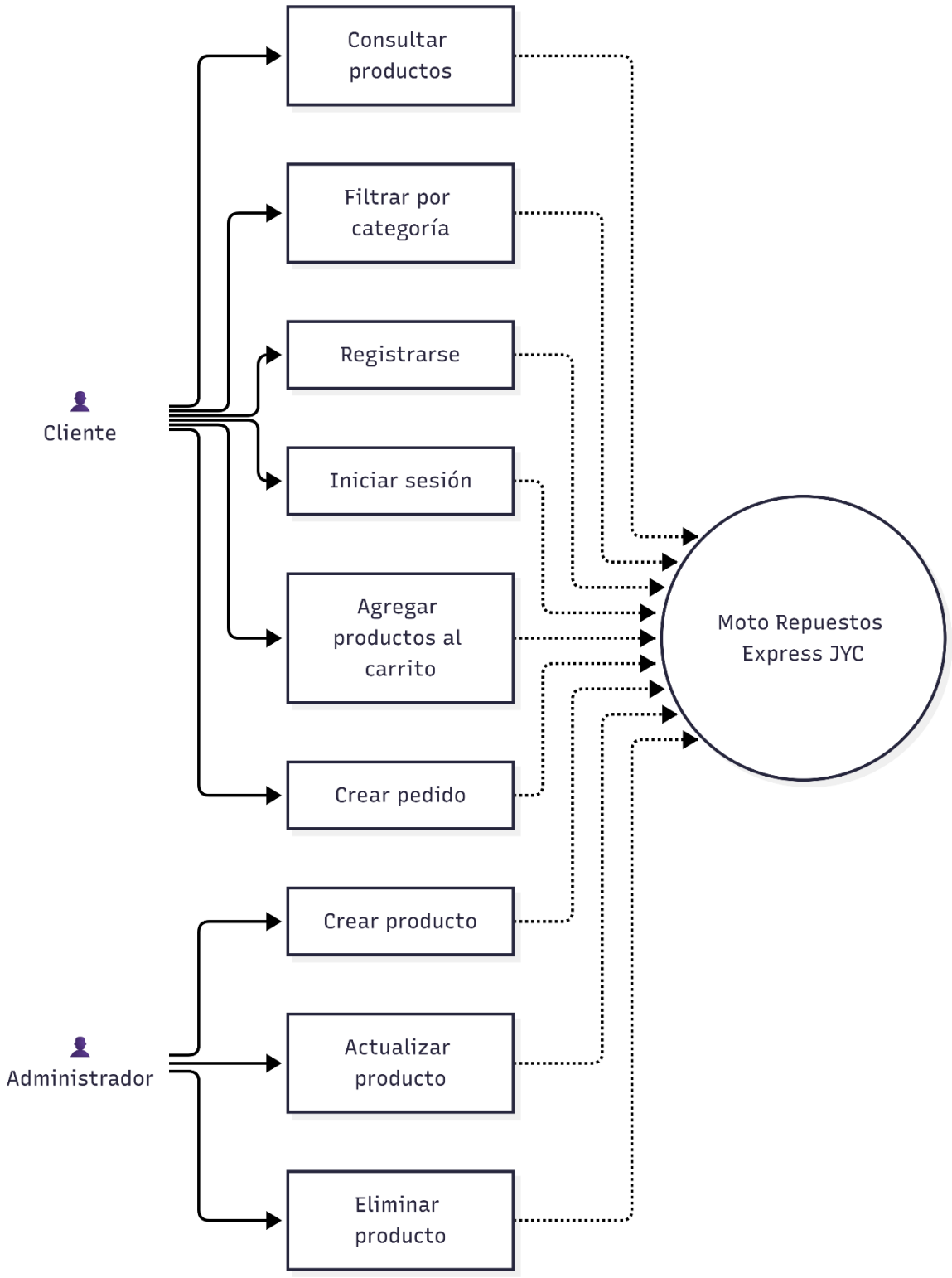


Diagrama de Clases y dependencias del Backend



El diagrama representa las entidades principales del sistema y las dependencias entre los controladores, servicios y repositorios del backend Spring Boot. Los controladores reciben las solicitudes HTTP, los servicios aplican las reglas de negocio y los repositorios gestionan la persistencia de la información. Un pedido contiene uno o varios detalles, y cada detalle se relaciona con un producto.

Diagrama de Clases y dependencias del Backend



Mapa de navegación de Moto Repuestos Express JYC

